

ServiYa - Documentacion Tecnica del Proyecto Final

ServiYa es una plataforma distribuida para gestionar solicitudes de servicios tecnicos, cotizaciones, pagos, comprobantes, postulacion de tecnicos, asignaciones, resenas, notificaciones y observabilidad operacional.

Esta documentacion esta preparada para MkDocs y GitHub Pages.

Enlaces de entrega

Recurso	Enlace
Repositorio GitHub	https://github.com/mary321mq/serviya-v2
Documentacion publicada	https://mary321mq.github.io/serviya-v2/
Entregables S15	Entregables finales
Presentacion PPTX	ProductoCurso_ServiYa_Presentacion.pptx

Objetivo del sistema

Permitir que un cliente solicite un servicio, reciba una cotizacion, pague, seleccione o reciba asignacion de un tecnico, confirme la finalizacion del trabajo y emita una resena.

Arquitectura general

- Frontend Angular: `serviya-web`.
- API Gateway: entrada unica para clientes web.
- Keycloak: autenticacion, autorizacion y roles.

- Config Server: configuracion centralizada.
- Eureka Server: descubrimiento de servicios.
- Microservicios: `user-ms`, `service-request-ms`, `technician-ms`, `assignment-ms`, `payment-ms`, `review-ms`, `notification-ms`.
- Observabilidad: Prometheus, Grafana, Loki y Promtail.

Organizacion

- Unidad 1 (U1): artefactos S01 a S05.
- Unidad 2 (U2): artefactos S06 a S12.
- Unidad 3 (U3): validacion, estabilizacion y defensa tecnica S13 a S15.
- Defensa S15: entregables finales, demo, rubrica, evidencia GitHub y topics.
- Anexos Individuales: evidencia personal de cada integrante.

Ejecucion local de la documentacion

```
pip install -r requirements-docs.txt  
mkdocs serve
```



Abrir:

```
http://localhost:8000
```



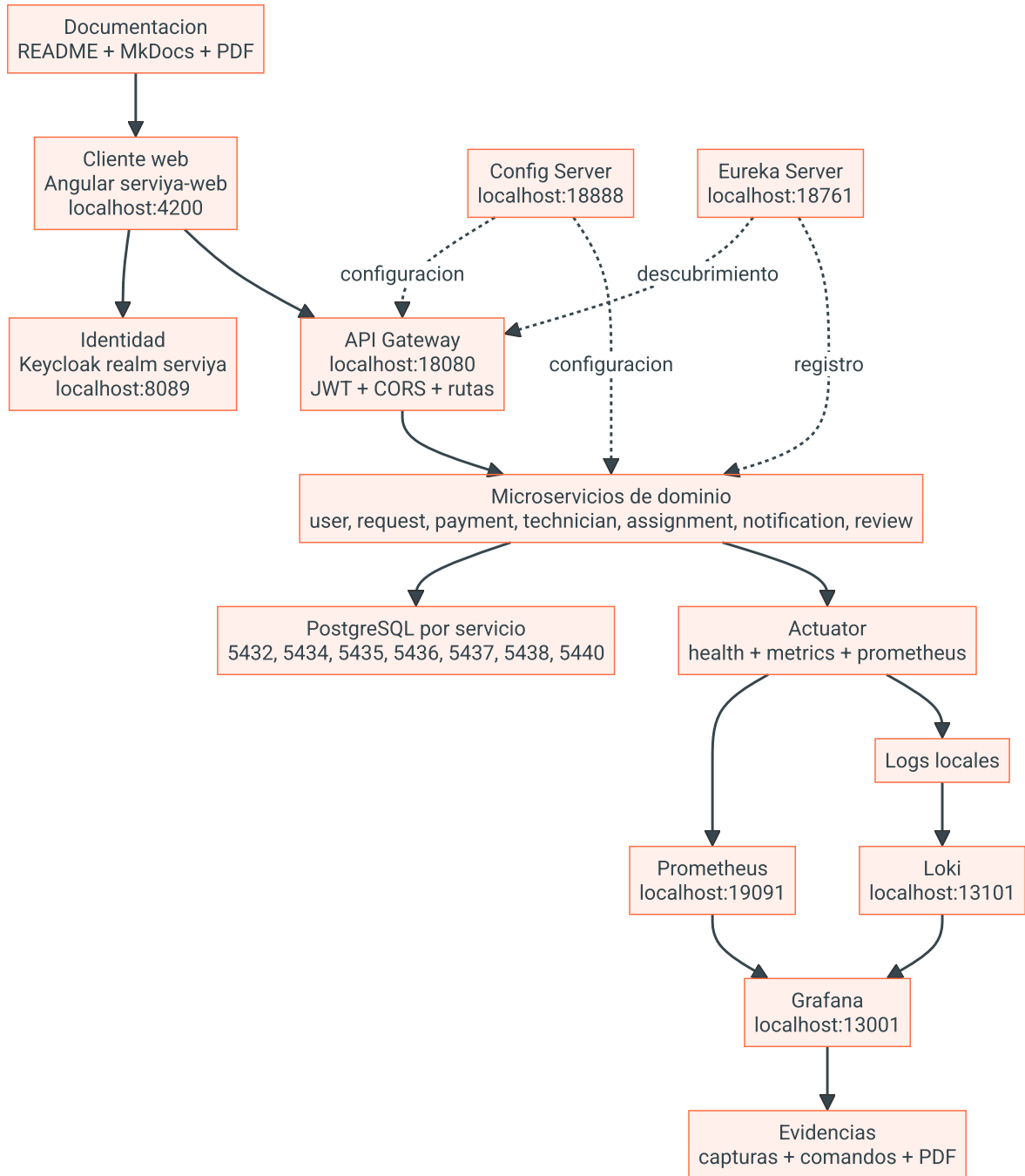
Arquitectura del producto ServiYa v2

ServiYa v2 es una plataforma distribuida para gestionar servicios tecnicos: registro de usuarios, postulacion de tecnicos, catalogo de servicios, solicitudes, asignaciones, pagos, billetera, resenas, notificaciones y monitoreo. El producto esta separado en una aplicacion Angular, una capa de infraestructura y microservicios Spring Boot con bases de datos PostgreSQL independientes.

Proposito de la arquitectura

La arquitectura busca que cada capacidad del negocio pueda evolucionar de forma separada. El frontend consume una unica entrada HTTP mediante el API Gateway, la seguridad se centraliza con Keycloak, la configuracion se publica desde Config Server y los servicios se registran en Eureka. Para la defensa tecnica, esto permite explicar el sistema completo por capas y no como una sola aplicacion monolitica.

Vista general de contenedores



Capas del sistema

Capa	Componentes	Responsabilidad
Presentacion	clients/serviya-web	Interfaz web por rol, rutas protegidas, consumo del gateway y pantallas de operacion.
Entrada HTTP	infra/gateway	Enrutamiento por prefijo, CORS, validacion JWT y control de acceso por rol.
Identidad	infra/keycloak	Login, emision de tokens, realm <code>serviya</code> , roles y usuarios de prueba.
Configuracion	infra/config + infra/config/config-repo	Propiedades centralizadas para servicios, seguridad, datasource, Eureka y observabilidad.
Descubrimiento	infra/eureka	Registro de servicios y consulta de instancias disponibles.
Dominio	services/*	Reglas de negocio separadas por contexto: usuarios, solicitudes, pagos, tecnicos, asignaciones, notificaciones y reseñas.
Datos	PostgreSQL por servicio	Persistencia aislada por microservicio para reducir acoplamiento.
Observabilidad	Prometheus, Grafana, Loki, Promtail	Metricas, salud, latencia, errores y logs centralizados para diagnostico.

Microservicios y datos

Servicio	Ruta	Puerto app	Base de datos	Puerto DB	Responsabilidad principal
user-ms	services/ user-ms	18082	serviya_u ser	5432	Usuarios, perfil, administracion de cuentas e

Servicio	Ruta	Puerto app	Base de datos	Puerto DB	Responsabilidad principal
					integracion con Keycloak.
service-request-ms	services/service-request-ms	8084	service_request_ms	5434	Solicitudes, catalogo operativo, estados, cotizaciones y cierre del servicio.
payment-ms	services/payment-ms	8083	serviayment	5435	Pagos, comprobantes, billetera del tecnico y liquidaciones.
tecniciamn-ms	services/tecniciamn-ms	8085	tecniciamn-ms	5436	Postulacion, documentos, especialidades, disponibilidad, ubicacion y portafolio.
assignment-ms	services/assignment-ms	8086	assignment-ms	5440	Asignaciones y ofertas relacionadas con tecnicos.
notification-ms	services/notification-ms	8087	notification_ms	5438	Notificaciones internas por usuario y eventos del sistema.
review-ms	services/reviews-	8088	review_ms	5437	Calificaciones, respuestas y

Servicio	Ruta	Puerto app	Base de datos	Puerto DB	Responsabilidad principal
	ms				moderacion de resenas.

Rutas publicadas por el Gateway

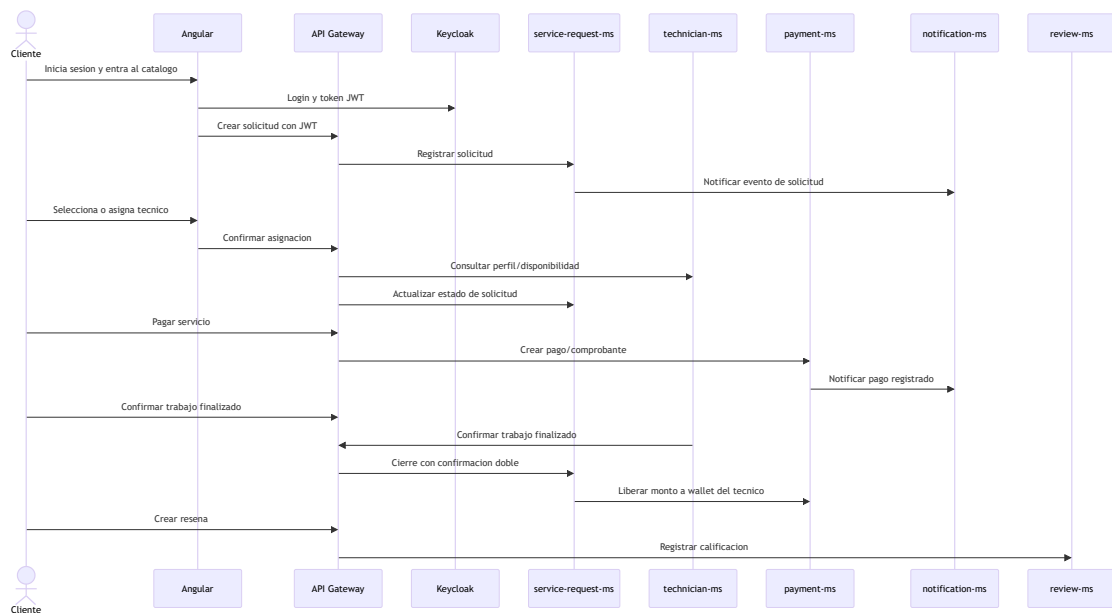
El gateway expone una ruta por microservicio usando prefijos estables. Cada ruta aplica `StripPrefix=1`, por lo que el microservicio recibe la ruta interna sin el prefijo.

Prefijo externo	Servicio destino	Uso esperado
/user-ms/**	user-ms	Usuarios, perfiles, admin de usuarios.
/service-request-ms/**	service-request-ms	Catalogo, solicitudes y estados del servicio.
/payment-ms/**	payment-ms	Checkout, pagos, historial, wallet y liquidaciones.
/technician-ms/**	technician-ms	Postulacion, documentos, perfil tecnico y disponibilidad.
/assignment-ms/**	assignment-ms	Asignaciones y ofertas.
/notification-ms/**	notification-ms	Bandeja de notificaciones.
/review-ms/**	review-ms	Resenas de clientes, tecnicos y administracion.

Modulos visibles en el frontend

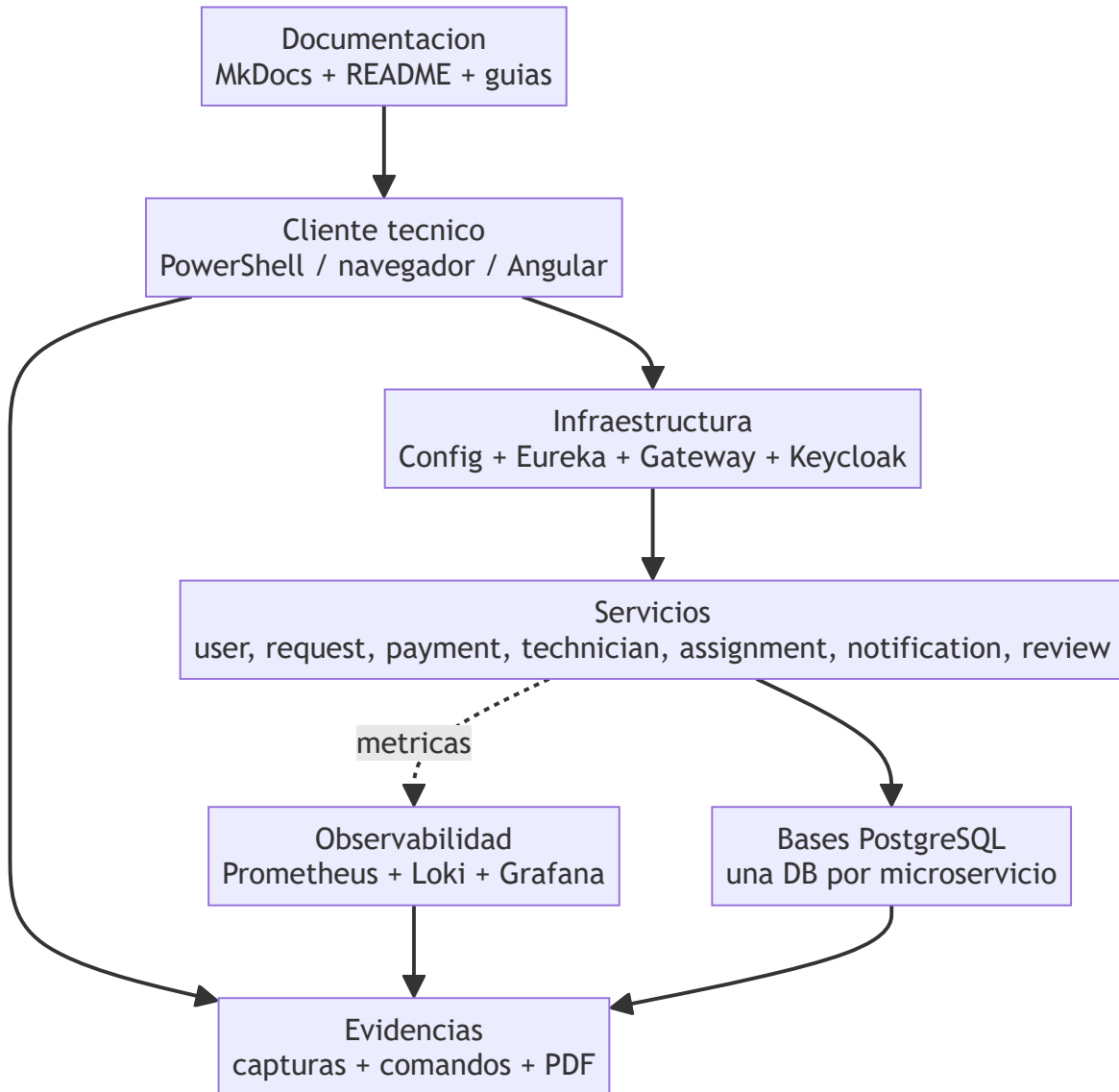
Rol	Rutas principales	Funcionalidad visible
Cliente	<code>/cliente</code> , <code>/cliente/servicios</code> , <code>/cliente/solicitudes</code> , <code>/cliente/checkout</code> , <code>/cliente/pagos</code> , <code>/cliente/resenas</code>	Buscar servicios, crear solicitudes, asignar tecnico, pagar, confirmar finalizacion y calificar.
Tecnico	<code>/tecnico</code> , <code>/tecnico/postulacion</code> , <code>/tecnico/documentos</code> , <code>/tecnico/ofertas</code> , <code>/tecnico/servicios</code> , <code>/tecnico/wallet</code> , <code>/tecnico/resenas</code>	Postular, completar perfil, responder ofertas, confirmar trabajo, revisar billetera y responder resenas.
Trabajador	<code>/trabajador</code>	Operacion interna segun rol asignado.
Admin	<code>/admin</code> , <code>/admin/usuarios</code> , <code>/admin/catalogo/servicios</code> , <code>/admin/monitor</code> , <code>/admin/pagos</code> , <code>/admin/reportes</code>	Administracion, monitoreo, catalogos, pagos, solicitudes, reportes y configuracion.

Flujo funcional principal



Revision tecnica en DEV

Esta vista resume como se valida el producto durante desarrollo local. Se parece al flujo que se pide en las sesiones de cierre tecnico: documentacion, cliente, infraestructura, servicios, observabilidad y evidencias.

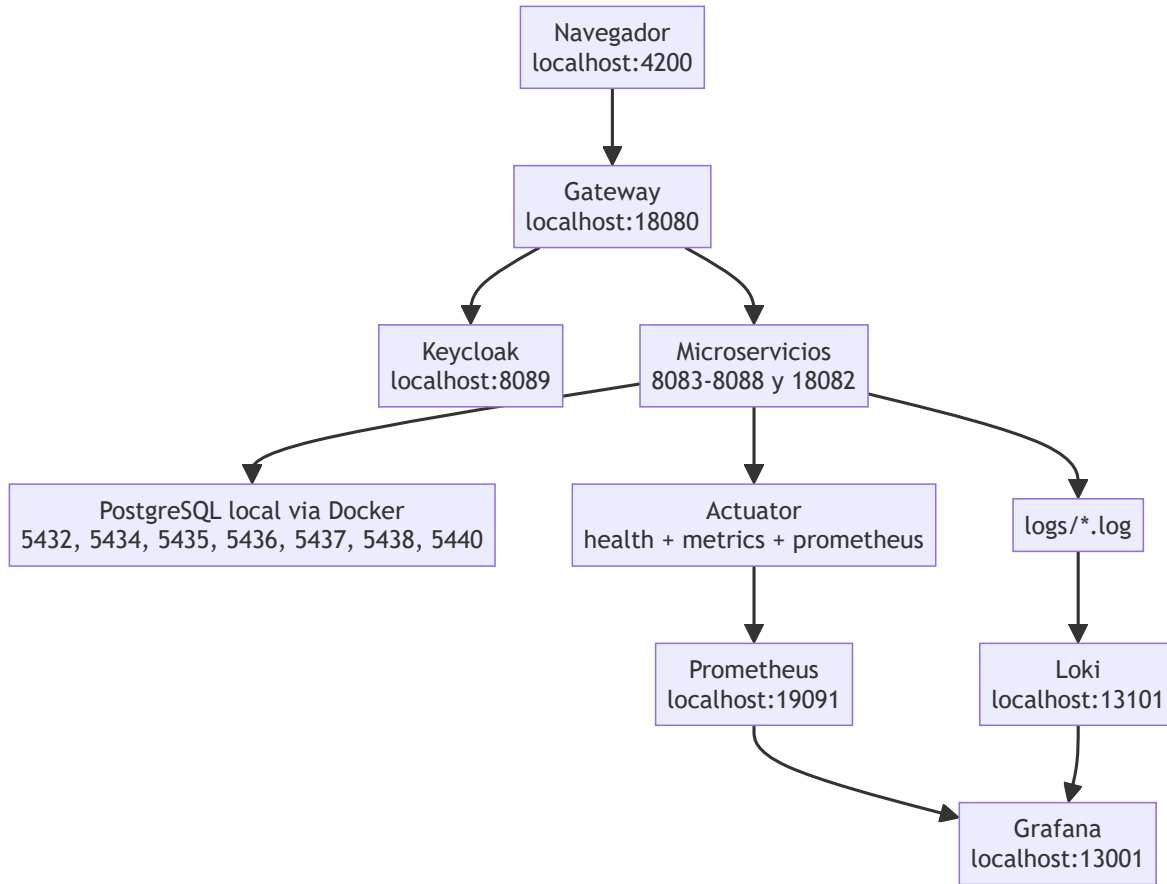


Checklist DEV:

Paso	Evidencia esperada
Levantar Keycloak y bases PostgreSQL	Contenedores <code>serviya-*</code> activos en Docker.
Levantar Config Server, Eureka y Gateway	<code>health=UP</code> y servicios visibles en Eureka.
Levantar microservicios	Endpoints <code>/actuator/health</code> responden <code>UP</code> .
Abrir Angular	<code>http://localhost:4200</code> muestra login y modulos por rol.
Ejecutar flujo cliente-tecnico	Solicitud creada, pago registrado, doble confirmacion y resena.
Revisar observabilidad	Prometheus con targets activos y Grafana con metricas.

Revision tecnica en PROD local

Para una defensa local, la maquina funciona como entorno de integracion: Docker aloja dependencias, Maven ejecuta servicios Spring Boot y Angular corre el cliente web. No es nube productiva, pero si permite demostrar el producto completo con infraestructura real.



Puntos tecnicos para sustentar

- Separacion por contexto: cada microservicio tiene responsabilidad, puerto y base de datos propia.
- Seguridad centralizada: Angular obtiene token de Keycloak y el gateway/servicios validan JWT del realm `serviya`.
- Comunicacion controlada: el frontend no llama directo a cada servicio, entra por `api-gateway`.
- Configuracion reproducible: los YAML centralizados viven en `infra/config/config-repo`.
- Diagnostico: Actuator expone `health`, `metrics` y `prometheus`; Grafana muestra estado, latencia y errores.
- Flujo de negocio completo: solicitud, tecnico, pago, confirmacion doble, billetera, resena y notificacion.

Manual de ejecucion local

Esta guia permite levantar ServiYa v2 desde cero en Windows con PowerShell. La ruta usada en esta maquina es:

```
$ROOT = "C:\Users\ACER\Downloads\nuestro sistema\serviya-v2\serviya-v2"  
cd $ROOT
```

Si el proyecto se clona en otra carpeta, cambia solo el valor de `$ROOT`.

Requisitos

Herramienta	Uso
Java 17	Ejecutar servicios Spring Boot.
Maven 3.9+	Compilar el proyecto padre, librerias y microservicios.
Node.js + npm	Instalar dependencias y levantar Angular.
Docker Desktop	Ejecutar Keycloak, PostgreSQL, Prometheus, Loki y Grafana.
Git	Clonar, versionar y subir cambios a GitHub.
PowerShell	Ejecutar los comandos de arranque y validacion.

1. Preparar el proyecto Maven

Desde la raiz del repositorio:

```
cd $ROOT  
mvn install -N  
mvn -pl libs/testing-support,libs/commons-web install -DskipTests
```

Esto instala el POM padre y las librerías comunes que necesitan los microservicios.

2. Levantar infraestructura Docker

Keycloak:

```
cd $ROOT\infra\keycloak
docker compose -f compose-dev.yml up -d
```

Bases de datos por microservicio:

```
cd $ROOT\services\user-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\service-request-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\payment-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\technician-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\assignment-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\notification-ms; docker compose -f compose-dev.yml up -d
cd $ROOT\services\reviews-ms; docker compose -f compose-dev.yml up -d
```

Observabilidad:

```
cd $ROOT\infra\observability
docker compose -f compose-dev.yml up -d
```

Verificación rápida de contenedores:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

3. Levantar infraestructura Spring

Abre una terminal por proceso para que cada servicio quede activo.

Config Server:

```
cd $ROOT\infra\config
mvn spring-boot:run
```

Eureka:

```
cd $ROOT\infra\eureka
mvn spring-boot:run
```

API Gateway:

```
cd $ROOT\infra\gateway
mvn spring-boot:run
```

4. Levantar microservicios

Ejecuta cada microservicio en su propia terminal:

```
cd $ROOT\services\user-ms; mvn spring-boot:run
cd $ROOT\services\payment-ms; mvn spring-boot:run
cd $ROOT\services\service-request-ms; mvn spring-boot:run
cd $ROOT\services\technician-ms; mvn spring-boot:run
cd $ROOT\services\assignment-ms; mvn spring-boot:run
cd $ROOT\services\notification-ms; mvn spring-boot:run
cd $ROOT\services\reviews-ms; mvn spring-boot:run
```

5. Levantar frontend Angular

```
cd $ROOT\clients\serviya-web
npm install
npm start -- --host 0.0.0.0 --port 4200
```

Abrir en navegador:

```
http://localhost:4200
```

Puertos de trabajo

Componente	URL
Frontend Angular	http://localhost:4200
API Gateway	http://localhost:18080
Config Server	http://localhost:18888
Eureka	http://localhost:18761

Componente	URL
Keycloak	http://localhost:8089
Grafana	http://localhost:13001
Prometheus	http://localhost:19091
Loki	http://localhost:13101
user-ms	http://localhost:18082/actuator/health
payment-ms	http://localhost:8083/actuator/health
service-request-ms	http://localhost:8084/actuator/health
technician-ms	http://localhost:8085/actuator/health
assignment-ms	http://localhost:8086/actuator/health
notification-ms	http://localhost:8087/actuator/health
review-ms	http://localhost:8088/actuator/health

Bases de datos locales

Servicio	Contenedor	DB	Puerto host
user-ms	serviya-postgres-user	serviya_user	5432
service-request-ms	serviya-postgres-service-request	service_request_ms	5434
payment-ms	serviya-postgres-payment	serviya_payment	5435

Servicio	Contenedor	DB	Puerto host
technician-ms	serviya-postgres-technician	serviya_technician	5436
assignment-ms	serviya-postgres-assignment	assignment_ms	5440
notification-ms	serviya-postgres-notification	notification_ms	5438
review-ms	serviya-postgres-review	review_ms	5437

Validacion de salud

Cuando todo este levantado, ejecuta:

```
$checks = @(
    "http://localhost:18888/actuator/health",
    "http://localhost:18761/actuator/health",
    "http://localhost:18080/actuator/health",
    "http://localhost:18082/actuator/health",
    "http://localhost:8083/actuator/health",
    "http://localhost:8084/actuator/health",
    "http://localhost:8085/actuator/health",
    "http://localhost:8086/actuator/health",
    "http://localhost:8087/actuator/health",
    "http://localhost:8088/actuator/health"
)

foreach ($url in $checks) {
    try {
        $r = Invoke-RestMethod -Uri $url -TimeoutSec 5
        "OK $url $($r.status)"
    } catch {
        "ERR $url $($_.Exception.Message)"
    }
}
```

Validacion funcional sugerida

1. Abrir `http://localhost:4200`.

2. Iniciar sesion con un usuario del realm `serviya`.
3. Entrar como cliente y crear una solicitud desde el catalogo.
4. Asignar o aceptar un tecnico.
5. Registrar pago desde checkout.
6. Confirmar trabajo terminado desde cliente y tecnico.
7. Revisar que el pago se libere a la wallet del tecnico.
8. Crear una resena del servicio.
9. Revisar notificaciones y dashboard de observabilidad.

Problemas frecuentes

Sintoma	Causa probable	Solucion
<code>npm.ps1 cannot be loaded</code>	Politica de ejecucion de PowerShell.	Usar <code>npm.cmd install</code> y <code>npm.cmd start - --host 0.0.0.0 --port 4200</code> , o ajustar la politica si el equipo lo permite.
<code>Port 5432 already in use</code>	PostgreSQL local ocupa el puerto de <code>user-ms</code> .	Detener el PostgreSQL local o cambiar el puerto publicado del compose de <code>user-ms</code> y actualizar <code>SERVIYA_DATASOURCE_URL</code> .
Servicios no aparecen en Eureka	Eureka o Config Server no estan levantados.	Iniciar primero Config Server, luego Eureka, luego Gateway y microservicios.
<code>401</code> o <code>403</code> en frontend	Token inexistente, vencido o rol incorrecto.	Cerrar sesion, iniciar de nuevo y validar roles en Keycloak.
Prometheus no muestra targets	Servicio apagado o sin <code>/actuator/prometheus</code> .	Verificar <code>/actuator/health</code> y revisar <code>infra/observability/prometheus/prometheus-dev.yml</code> .
GitHub Pages sale rojo	Pages no esta usando GitHub Actions o fallo MkDocs.	Revisar <code>Settings > Pages</code> y el workflow <code>Deploy MkDocs</code> .

Comandos de documentacion

Servir MkDocs localmente:

```
cd $ROOT  
mkdocs serve
```



Construir documentacion:

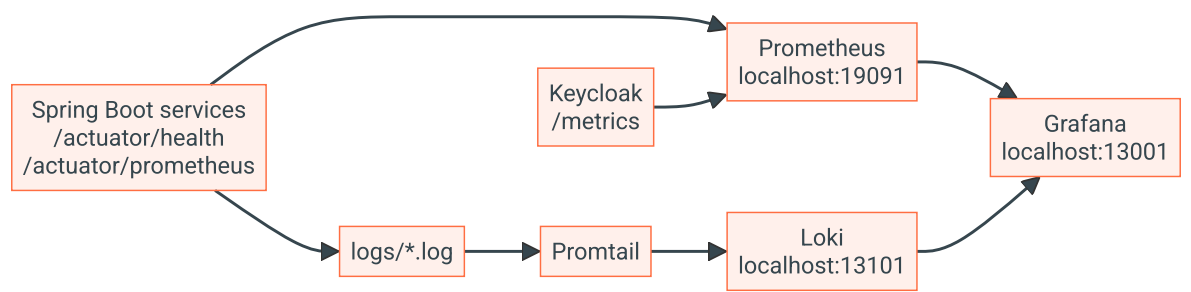
```
cd $ROOT  
mkdocs build --strict
```



Observabilidad y diagnostico

ServiYa v2 incorpora observabilidad para que la defensa tecnica no dependa solo de pantallas. La plataforma expone salud, metricas HTTP, latencia, errores y logs mediante Actuator, Prometheus, Loki, Promtail y Grafana.

Arquitectura de observabilidad



URLs locales

Herramienta	URL	Uso en defensa
Prometheus	<code>http://localhost:19091</code>	Revisar targets, consultas <code>up</code> , requests y latencia.
Grafana	<code>http://localhost:13001</code>	Mostrar dashboard de salud, trafico y errores.
Loki	<code>http://localhost:13101</code>	Almacenar logs consultables desde Grafana.
Eureka	<code>http://localhost:18761</code>	Confirmar registro de servicios.

Credenciales Grafana en desarrollo:

```
usuario: admin
password: admin
```



Targets esperados en Prometheus

Prometheus toma metricas desde `infra/observability/prometheus/prometheus-dev.yml`.

Target	Aplicacion
<code>host.docker.internal:18888</code>	<code>config-server</code>
<code>host.docker.internal:18761</code>	<code>eureka-server</code>
<code>host.docker.internal:18080</code>	<code>api-gateway</code>
<code>host.docker.internal:18082</code>	<code>user-ms</code>
<code>host.docker.internal:8083</code>	<code>payment-ms</code>
<code>host.docker.internal:8084</code>	<code>service-request-ms</code>
<code>host.docker.internal:8085</code>	<code>technician-ms</code>
<code>host.docker.internal:8086</code>	<code>assignment-ms</code>
<code>host.docker.internal:8087</code>	<code>notification-ms</code>
<code>host.docker.internal:8088</code>	<code>review-ms</code>
<code>host.docker.internal:8089</code>	<code>keycloak</code>

Que debe mostrar Grafana

Seccion	Paneles	Interpretacion
Salud general	Estado UP/DOWN por servicio	Permite detectar servicios caidos durante la demo.
Trafico HTTP	Requests por segundo por aplicacion	Demuestra que el frontend esta generando llamadas reales.
Latencia	Percentil p95	Identifica servicios lentos o endpoints pesados.
Errores	Conteo 4xx/5xx	Separa errores de permisos/rutas de fallos internos.
JVM	CPU y memoria	Ayuda a sustentar estabilidad del backend.
Logs	Entradas por servicio	Permite revisar trazas y excepciones.

Consultas PromQL utiles

Estado de targets:

```
up
```



Requests por segundo:

```
sum by (application) (rate(http_server_requests_seconds_count[5m]))
```



Errores 4xx y 5xx:

```
sum by (application, status)
(rate(http_server_requests_seconds_count{status=~"4..|5.."}[5m]))
```



Latencia p95:

```
histogram_quantile(0.95, sum by (le, application)
(rate(http_server_requests_seconds_bucket[5m])))
```



Pasos de diagnostico durante la defensa

1. Abrir Eureka y confirmar que los servicios estan registrados.
2. Abrir Prometheus > Status > Targets y verificar que los targets esten `UP`.
3. Abrir Grafana y mostrar el dashboard general.
4. Ejecutar una accion en Angular, por ejemplo crear o consultar una solicitud.
5. Volver a Grafana y mostrar que suben los requests del gateway y del microservicio correspondiente.
6. Si ocurre un error, revisar si es `401/403`, `404`, `5xx` o si un servicio esta caido.

Diagnostico por sintomas

Sintoma	Donde mirar	Posible causa
Frontend carga pero no trae datos	Consola navegador, Gateway, Grafana	Token invalido, ruta incorrecta o gateway apagado.
Servicio no aparece en Grafana	Prometheus Targets	Servicio no levantado o Actuator no expuesto.
<code>401 / 403</code>	Keycloak, token JWT, roles	Usuario sin rol requerido o sesion vencida.
<code>5xx</code>	Logs del microservicio y Loki	Excepcion interna, base caida o dependencia no disponible.
Alta latencia	Panel p95 y logs	Base de datos lenta, llamada Feign o endpoint pesado.

Evidencias para entregar

- Captura de Grafana con todos los servicios esperados.
- Captura de Prometheus Targets en estado `UP`.
- Captura de Eureka con microservicios registrados.

- Captura del flujo funcional en Angular.
- PDF exportado desde MkDocs con esta guía y la arquitectura del producto.

Walkthrough funcional del sistema

Este walkthrough muestra el recorrido completo que se debe demostrar en la defensa tecnica. El objetivo es que el jurado vea que ServiYa no solo tiene pantallas, sino un flujo distribuido donde intervienen frontend, gateway, seguridad, microservicios, bases de datos, eventos y observabilidad.

Actores

Actor	Modulo Angular	Acciones principales
Cliente	<code>/cliente</code>	Buscar servicios, crear solicitudes, pagar, confirmar cierre y calificar.
Tecnico	<code>/tecnico</code>	Postular, completar perfil, recibir ofertas, atender servicios, confirmar cierre y revisar wallet.
Trabajador	<code>/trabajador</code>	Apoyo operativo segun permisos asignados.
Admin	<code>/admin</code>	Gestion de usuarios, tecnicos, catalogo, pagos, solicitudes, reportes y monitoreo.

Recorrido 1: ingreso seguro

1. El usuario abre `http://localhost:4200`.
2. Angular redirige al login cuando no existe sesion activa.
3. Keycloak autentica al usuario y emite un JWT del realm `serviya`.
4. El frontend aplica guardias de autenticacion y rol.
5. El gateway y los microservicios validan el token antes de procesar endpoints protegidos.

Evidencia sugerida:

- Captura de login.

- Captura de ruta protegida por rol.
- Captura de respuesta `401/403` cuando se intenta entrar sin permisos.

Recorrido 2: cliente crea una solicitud

1. El cliente entra al catalogo en `/cliente/servicios`.
2. Selecciona un servicio y revisa el detalle.
3. Crea una solicitud desde `/cliente/solicitudes/nueva`.
4. Angular envia la peticion al gateway.
5. El gateway enruta hacia `service-request-ms`.
6. `service-request-ms` guarda la solicitud en `service_request_ms`.
7. `notification-ms` registra o expone notificaciones relacionadas al evento.

Resultado esperado:

Elemento	Resultado
Frontend	La solicitud aparece en <code>/cliente/solicitudes</code> .
Backend	<code>service-request-ms</code> responde correctamente.
Datos	Existe registro en la base <code>service_request_ms</code> .
Observabilidad	Aparecen requests en Prometheus/Grafana para gateway y servicio.

Recorrido 3: tecnico postula y atiende

1. Un usuario con rol tecnico entra a `/tecnico/postulacion`.
2. Completa datos, documentos, especialidades, disponibilidad y ubicacion.
3. El administrador puede revisar postulaciones desde `/admin/tecnicos/postulaciones`.
4. Cuando el tecnico esta habilitado, puede ver ofertas o asignaciones en `/tecnico/ofertas` y `/tecnico/servicios`.
5. Al aceptar o atender un servicio, el estado se actualiza mediante los servicios de `solicitud/asignacion`.

Evidencia sugerida:

- Perfil tecnico completo.
- Documento o postulacion visible.
- Oferta o servicio asignado.
- Estado actualizado en pantalla.

Recorrido 4: pago y comprobante

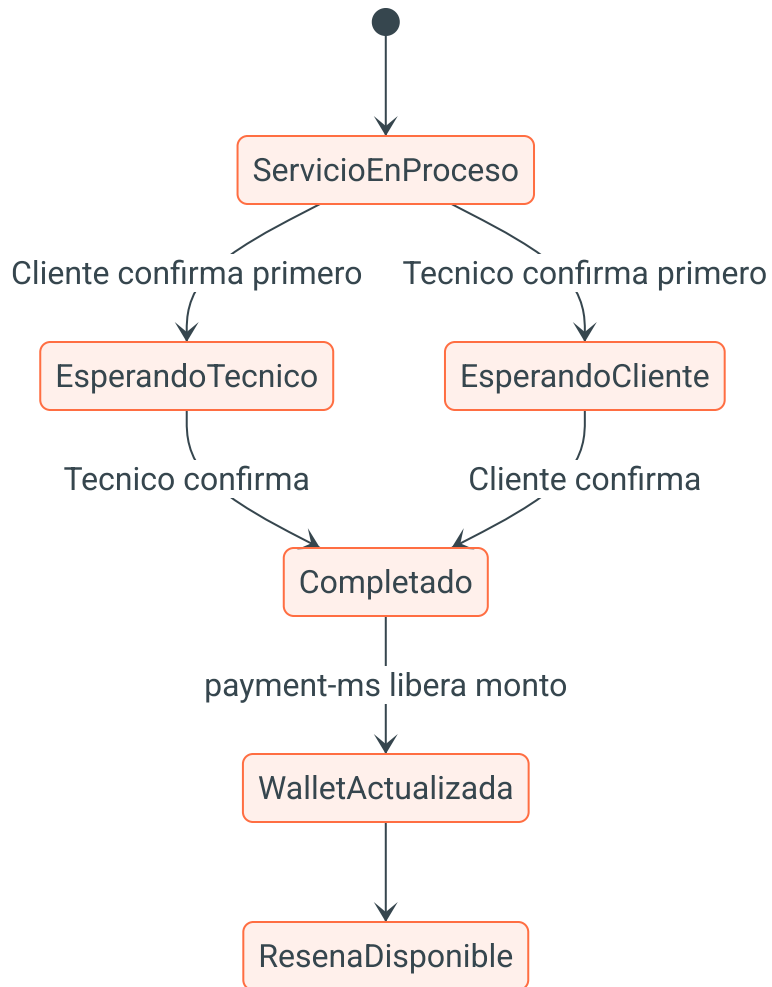
1. El cliente entra a checkout desde `/cliente/checkout`.
2. El frontend consume `/payment-ms/**` mediante el gateway.
3. `payment-ms` registra pago o intencion de pago en `serviya_payment`.
4. El cliente puede revisar historial en `/cliente/pagos`.
5. El administrador puede monitorear pagos desde `/admin/pagos` o reportes.

Evidencia sugerida:

Vista	Que mostrar
Cliente	Checkout y pagos realizados.
Admin	Monitor o historial de pagos.
Observabilidad	Requests a <code>payment-ms</code> y latencia del endpoint.

Recorrido 5: confirmacion doble y wallet

ServiYa implementa una regla importante del negocio: el trabajo no se cierra solo con una parte. Cliente y tecnico deben confirmar la finalizacion.



Resultado esperado:

- Si el cliente confirma primero, la pantalla queda esperando al tecnico.
- Si el tecnico confirma primero, la pantalla queda esperando al cliente.
- Solo cuando ambos confirman, la solicitud pasa a `COMPLETADO`.
- `payment-ms` libera el monto a la wallet del tecnico.
- El tecnico puede revisar su wallet en `/tecnico/wallet`.
- El cliente puede registrar una resena.

Recorrido 6: resenas y respuesta

1. Cuando el servicio termina, el cliente accede a `/cliente/resenas/nueva/:serviceRequestId`.
2. El gateway enruta hacia `review-ms`.

3. `review-ms` guarda la calificación en `review_ms`.
4. El tecnico revisa resenas en `/tecnico/resenas`.
5. El tecnico puede responder desde `/tecnico/resenas/:id/responder`.
6. El admin revisa moderacion o detalle desde `/admin/resenas`.

Recorrido 7: monitoreo operativo

Durante la demo, abrir:

Herramienta	URL	Que explicar
Eureka	<code>http://localhost:18761</code>	Servicios registrados y disponibles.
Grafana	<code>http://localhost:13001</code>	Estado, trafico, errores y latencia.
Prometheus	<code>http://localhost:19091</code>	Targets activos y consultas PromQL.
Keycloak	<code>http://localhost:8089</code>	Realm, usuarios y roles.

Consultas utiles en Prometheus:

```
up
```



```
sum by (application) (rate(http_server_requests_seconds_count[5m]))
```



```
histogram_quantile(0.95, sum by (le, application) (rate(http_server_requests_seconds_bucket[5m])))
```



Guion corto para sustentar

1. Presentar la arquitectura: Angular, Gateway, Keycloak, Config, Eureka, microservicios y observabilidad.
2. Mostrar login y rutas por rol.
3. Crear una solicitud como cliente.
4. Mostrar tecnico y asignacion/oferta.

5. Registrar pago.
6. Confirmar cierre desde ambas partes.
7. Mostrar wallet y resena.
8. Cerrar con Grafana/Prometheus para evidenciar salud, trafico y errores.

Documentación con MkDocs

El objetivo es instalar y configurar **MkDocs** en el proyecto para mantener un registro de toda la documentación (planes de implementación, arquitectura y manuales) de forma ordenada y fácil de leer.

User Review Required

[!IMPORTANT] Se instalará MkDocs utilizando Python/Pip y se agregará la dependencia `mkdocs-material` para utilizar un tema de diseño moderno y elegante (ampliamente usado en la industria). Por favor, confirma si estás de acuerdo con esto.

Proposed Changes

1. Instalación de Dependencias

Se ejecutará el siguiente comando para instalar las herramientas necesarias (usando `pip`):

```
pip install mkdocs mkdocs-material
```



2. Estructura de Documentación

Se inicializará el proyecto de MkDocs en la raíz del repositorio (`C:\ServiYa\serviya-v2`).

[NEW] `C:\ServiYa\serviya-v2\mkdocs.yml`

Archivo de configuración principal de MkDocs, especificando el nombre del proyecto ("ServiYa Docs") y el tema `material`.

[NEW] Directorio `C:\ServiYa\serviya-v2\docs`

Contendrá todos los archivos Markdown de la documentación: - `index.md` : Página de inicio del proyecto. - `plan_implementacion.md` : Contendrá el último plan de implementación y el historial de características (tomado del artifact que tenemos actualmente). - `walkthrough.md` : Copia del historial de ejecución y pruebas realizadas.

Verification Plan

Manual Verification

1. Instalaré las herramientas e inicializaré el proyecto.
2. Migraré el contenido de `implementation_plan.md` actual a los archivos de MkDocs.
3. Te indicaré cómo ejecutar el servidor local (`mkdocs serve`) para que puedas visualizar la documentación completa en vivo desde tu navegador en `http://localhost:8000`.

S01 - Vision del proyecto

Objetivo

Documentar la vision inicial, problema, usuarios y alcance de ServiYa.

Pendiente de completar

- Problema identificado.
- Propuesta de solucion.
- Usuarios objetivo.
- Alcance funcional inicial.
- Evidencias de la sesion.

S02 - Requerimientos y alcance

Objetivo

Registrar requerimientos funcionales, no funcionales y alcance del proyecto.

Pendiente de completar

- Historias de usuario.
- Reglas de negocio.
- Requerimientos no funcionales.
- Restricciones.
- Evidencias de validacion.

S03 - Arquitectura inicial

Objetivo

Presentar la primera propuesta de arquitectura distribuida para ServiYa.

Pendiente de completar

- Diagrama inicial.
- Servicios propuestos.
- Responsabilidades.
- Riesgos arquitectonicos.
- Decisiones tecnicas.

S04 - Modelo de datos y contratos

Objetivo

Documentar entidades principales, bases de datos y contratos REST iniciales.

Pendiente de completar

- Modelo de datos.
- Entidades por microservicio.
- Contratos REST.
- Payloads de ejemplo.
- Criterios de validacion.

S05 - Prototipo funcional

Objetivo

Registrar el prototipo inicial del frontend y los primeros flujos navegables.

Pendiente de completar

- Pantallas iniciales.
- Flujo cliente.
- Flujo tecnico.
- Evidencias visuales.
- Ajustes posteriores.

S06 - Microservicios base

Objetivo

Documentar la implementacion inicial de los microservicios principales.

Pendiente de completar

- Estructura por servicio.
- Puertos.
- Bases de datos.
- Endpoints iniciales.
- Evidencias de ejecucion.

S07 - Seguridad con Keycloak

Objetivo

Documentar autenticacion, autorizacion, roles y configuracion de Keycloak.

Pendiente de completar

- Realm.
- Cliente OIDC.
- Roles.
- Usuarios de prueba.
- Reglas de acceso por ruta.

S08 - API Gateway y comunicacion

Objetivo

Documentar el API Gateway, rutas, filtros y comunicacion entre servicios.

Pendiente de completar

- Rutas del gateway.
- Reglas de seguridad.
- Comunicacion REST.
- Feign Clients.
- Pruebas con endpoints.

S09 - Flujo de solicitudes

Objetivo

Documentar el ciclo de vida de una solicitud de servicio.

Pendiente de completar

- Estados.
- Flujo remoto.
- Flujo presencial.
- Cotizacion.
- Asignacion de tecnico.

S10 - Pagos y comprobantes

Objetivo

Documentar pagos, comprobantes PDF, boleta, factura e historial.

Pendiente de completar

- Pago con tarjeta.
- Pago Yape/Plin.
- Boleta por DNI.
- Factura por RUC.
- Descarga de PDF.

S11 - Observabilidad

Objetivo

Documentar Prometheus, Grafana, Loki y Promtail.

Pendiente de completar

- Servicios monitoreados.
- Dashboard.
- Metricas.
- Logs.
- Troubleshooting.

S12 - Integracion frontend-backend

Objetivo

Documentar la integracion entre Angular, Gateway y microservicios.

Pendiente de completar

- Servicios Angular.
- Rutas protegidas.
- Consumo de APIs.
- Manejo de errores.
- Evidencias de integracion.

S13 - Pruebas e integracion

Objetivo

Documentar pruebas funcionales, integracion y verificacion por flujo.

Pendiente de completar

- Casos de prueba.
- Resultados.
- Bugs encontrados.
- Correcciones.
- Evidencias.

S14 - Estabilizacion y hardening

Objetivo

Documentar estabilizacion, correccion de errores y preparacion de defensa.

Pendiente de completar

- Errores corregidos.
- Ajustes de seguridad.
- Ajustes de observabilidad.
- Build final.
- Checklist de defensa.

S15 - Defensa Tecnica

1. Arquitectura final y flujo end-to-end implementado

La arquitectura final de ServiYa usa frontend Angular, API Gateway, Keycloak, Config Server, Eureka Server, microservicios Spring Boot, base de datos por servicio y observabilidad con Prometheus, Grafana, Loki y Promtail.

Componente	Responsabilidad
<code>serviya-web</code>	Interfaz web para cliente, tecnico, trabajador y admin
<code>api-gateway</code>	Enrutamiento, seguridad y punto unico de entrada
<code>keycloak</code>	Autenticacion, usuarios y roles
<code>config-server</code>	Configuracion centralizada
<code>eureka-server</code>	Registro y descubrimiento de servicios
<code>service-request-ms</code>	Solicitudes, cotizaciones, estados y asignacion
<code>technician-ms</code>	Postulacion, perfil y disponibilidad del tecnico
<code>assignment-ms</code>	Busqueda de tecnicos disponibles por categoria y ubicacion
<code>payment-ms</code>	Pagos, comprobantes, historial y billetera
<code>review-ms</code>	Resenas y calificaciones
<code>notification-ms</code>	Notificaciones in-app

Flujo end-to-end

1. Cliente inicia sesion en Keycloak.
2. Cliente selecciona un servicio.
3. Si es COTIZACION + REMOTA , se crea gratis y pasa a PENDIENTE_EVALUACION .
4. Si es COTIZACION + PRESENCIAL , se crea en ESPERANDO_PAGO_VISITA .
5. Cliente paga la visita presencial.
6. Backend cambia a PENDIENTE_EVALUACION .
7. Trabajador envia cotizacion.
8. Solicitud pasa a COTIZADO_ESPERANDO_PAGO .
9. Cliente paga la cotizacion.
10. Solicitud pasa a PAGADO_BUSCANDO_TECNICO .
11. Cliente selecciona tecnico.
12. Tecnico acepta o rechaza.
13. Si acepta, queda BUSY hasta finalizar.
14. Cliente y tecnico confirman finalizacion.
15. Solicitud pasa a COMPLETADO .
16. payment-ms acredita la billetera del tecnico.
17. Cliente deja resena.

2. Seguridad, comunicacion, consistencia y observabilidad

Seguridad

El sistema usa Keycloak con JWT y roles:

Rol	Permisos
CLIENTE	Crear solicitudes, pagar, ver historial y calificar
TECNICO	Ver ofertas, aceptar, rechazar y completar trabajos
TRABAJADOR	Revisar solicitudes y emitir cotizaciones
ADMIN	Gestionar catalogo, usuarios, tecnicos, pagos y monitoreo

Comunicacion sincrona

- Angular consume el API Gateway.
- Gateway enruta a microservicios.
- Los microservicios usan REST y Feign Client.

Ejemplos:

- `service-request-ms` llama a `assignment-ms`.
- `service-request-ms` llama a `technician-ms` para disponibilidad.
- `service-request-ms` llama a `payment-ms` para billetera.
- Servicios llaman a `notification-ms` para notificaciones.

Comunicacion asincrona o desacoplada

Las notificaciones se manejan como un componente desacoplado. Si falla la notificacion, no debe romper el flujo principal de negocio.

Consistencia

El sistema aplica consistencia eventual entre estados de solicitud, billetera, disponibilidad de tecnico y notificaciones.

Reglas criticas:

- Un tecnico que rechaza una solicitud no vuelve a aparecer para esa misma solicitud.
- Un tecnico ocupado no aparece para otros clientes.
- La cotizacion remota no cobra visita.
- La cotizacion presencial cobra visita antes de evaluacion.

Observabilidad

Herramienta	Puerto	Uso
Prometheus	19091	Metricas
Grafana	13001	Dashboards

Herramienta	Puerto	Uso
Loki	3100	Logs
Promtail	interno	Recoleccion de logs

Paneles esperados:

- Estado de servicios.
- Trafico HTTP por segundo.
- Latencia HTTP p95.
- Errores HTTP 4xx/5xx.
- CPU del proceso.
- Memoria JVM.
- Logs por microservicio.

3. Orden de arranque de los servicios

1. Bases de datos.
2. Keycloak.
3. Config Server.
4. Eureka Server.
5. API Gateway.
6. Microservicios backend.
7. Observabilidad.
8. Frontend Angular.

4. Puertos y variables de entorno

Servicio	Puerto
Frontend Angular	4200

Servicio	Puerto
API Gateway	18080
Config Server	18888
Eureka Server	18761
Keycloak	8089
service-request-ms	8084
payment-ms	8083
technician-ms	8085
assignment-ms	8086
notification-ms	8087
review-ms	8088
PostgreSQL service-request-ms	5434
PostgreSQL payment-ms	5435
PostgreSQL technician-ms	5436
PostgreSQL review-ms	5437
PostgreSQL notification-ms	5438
PostgreSQL assignment-ms	5440

Variables principales:

Variable	Ejemplo
CONFIG_SERVER_URL	http://localhost:18888
SERVIYA_KEYCLOAK_ISSUER_URI	http://localhost:8089/realms/serviya
SERVIYA_SERVICE_REQUEST_MS_URI	http://localhost:8084
SERVIYA_PAYMENT_MS_URI	http://localhost:8083
SERVIYA_TECHNICIAN_MS_URI	http://localhost:8085
SERVIYA_ASSIGNMENT_MS_URI	http://localhost:8086
SERVIYA_NOTIFICATION_MS_URI	http://localhost:8087
SERVIYA_REVIEW_MS_URI	http://localhost:8088

5. Rutas y datos de prueba

Base externa:

http://localhost:18080



Crear solicitud presencial

POST /service-request-ms/api/v1/solicitudes



```
{
  "catalogServiceId": 6,
  "direccionFisica": "Puno, San Roman, Juliaca: Heroes de La Guerra del
Pacifico",
  "latitud": -15.4992,
  "longitud": -70.1333,
  "cantidad": 1,
  "evidenciaUrls": [
    "/api/v1/solicitudes/evidencia/fuga.jpg"
  ]
}
```



```
]
}
```

Resultado esperado:

```
{
  "estadoSolicitud": "ESPERANDO_PAGO_VISITA",
  "costoVisita": 20.00
}
```

Crear solicitud remota

```
{
  "catalogServiceId": 7,
  "direccionFisica": "Puno, San Roman, Juliaca: Jr. Lima 123",
  "latitud": -15.5001,
  "longitud": -70.1299,
  "cantidad": 1,
  "evidenciaUrls": [
    "/api/v1/solicitudes/evidencia/problema-remoto.jpg"
  ]
}
```

Resultado esperado:

```
{
  "estadoSolicitud": "PENDIENTE_EVALUACION",
  "costoVisita": 0.00
}
```

Marcar pago

POST /service-request-ms/api/v1/solicitudes/{id}/marcar-pagado

Cotizar solicitud

POST /service-request-ms/api/v1/solicitudes/trabajador/{id}/cotizar

```
{
  "items": [
    {
      "descripcion": "Mano de obra",
      "cantidad": 1,

```

```
    "precioUnitario": 80.00
  }
]
}
```

Descargar comprobante

GET /payment-ms/api/v1/pagos/{id}/comprobante



6. Comandos de ejecucion

Bases de datos:

```
cd C:\ServiYa\serviya-v2\services\service-request-ms
docker compose -f compose-dev.yml up -d
```



Backend:

```
cd C:\ServiYa\serviya-v2\infra\config
mvn spring-boot:run
```



```
cd C:\ServiYa\serviya-v2\infra\eureka
mvn spring-boot:run
```



```
cd C:\ServiYa\serviya-v2\infra\gateway
mvn spring-boot:run
```



```
cd C:\ServiYa\serviya-v2\services\service-request-ms
mvn spring-boot:run
```



Frontend:

```
cd C:\ServiYa\serviya-v2\clients\serviya-web
npm install
ng serve
```



7. Evidencias esperadas y criterios de verificacion

Evidencia	Criterio
Login con Keycloak	Usuario autenticado con rol correcto
Solicitud remota	PENDIENTE_EVALUACION , costo visita 0
Solicitud presencial	ESPERANDO_PAGO_VISITA , costo visita mayor a 0
Pago de visita	Estado cambia a PENDIENTE_EVALUACION
Cotizacion enviada	Estado COTIZADO_ESPERANDO_PAGO
Pago de cotizacion	Estado PAGADO_BUSCANDO_TECNICO
Tecnico acepta	Estado EN_PROCESO , tecnico BUSY
Trabajo completado	Estado COMPLETADO
Billetera	Abono registrado
PDF	Comprobante descargable
Grafana	Dashboard con metricas y logs

8. Errores frecuentes y solucion

Puerto ocupado

```
netstat -ano | findstr :8085
taskkill /PID <PID> /F
```



@PathVariable sin nombre

Usar siempre:

```
@PathVariable("id") Long id
```



Cotizacion presencial no crea solicitud

Verificar que PostgreSQL permita `ESPERANDO_PAGO_VISITA` en `solicitudes_servicio_estado_solicitud_check`.

Prometheus muestra servicios abajo

Verificar:

```
http://localhost:8084/actuator/prometheus  
http://localhost:8083/actuator/prometheus  
http://localhost:8085/actuator/prometheus
```



Entregables BL

La defensa tecnica S15 exige tres entregables grupales. Este archivo deja trazabilidad de lo que debe estar en BL y de lo que queda evidenciado en GitHub.

Resumen de entrega

Entregable	Destino oficial	Evidencia en repo
Documentacion MkDocs	GitHub Pages y BL si se solicita enlace	docs/ , mkdocs.yml , .github/workflows/docs.yml
PDF grupal desde documentacion	BLearning (BL)	docs/defensa-s15/entregables- finales/S15_ServiYa_U3_Docs.pdf
Presentacion final	BLearning (BL)	docs/defensa-s15/entregables- finales/ProductoCurso_ServiYa_Presen tacion.pptx

1. Sitio de documentacion

Debe estar en el repositorio y publicado como sitio navegable.

- Repositorio: <https://github.com/mary321mq/serviya-v2>
- Sitio esperado de GitHub Pages: <https://mary321mq.github.io/serviya-v2/>
- Workflow: `.github/workflows/docs.yml`

Comando local:

```
cd "C:\Users\ACER\Downloads\nuestro sistema\serviya-v2\serviya-v2"  
pip install -r requirements-docs.txt  
mkdocs serve
```

URL local:

http://localhost:8000



Publicacion en GitHub Pages

1. Entrar a GitHub, seccion `Settings` -> `Pages` .
2. En `Build and deployment` , seleccionar `GitHub Actions` .
3. Ejecutar el workflow `Deploy MkDocs` o hacer push a `main` .
4. Verificar el enlace publicado.

2. PDF grupal desde la documentacion

Nombre solicitado por la guia del curso:

S15_Equipo##_U3_Docs.pdf



Forma aceptada:

1. Abrir el sitio MkDocs publicado o local.
2. Usar imprimir del navegador.
3. Guardar como PDF.
4. Subir a BL.

Regla importante

El PDF debe generarse desde la documentacion. No se acepta un PDF armado manualmente fuera del sitio.

3. Presentacion final

Archivo disponible en el repo:

docs/defensa-s15/entregables-finales/ProductoCurso_ServiYa_Presentacion.pptx



Nombre solicitado por la guia del curso si se exporta como PDF:

ProductoCurso_Equipo##_Presentacion.pdf



Contenido minimo esperado:

- Nombre del proyecto y equipo.
- Problema o flujo de negocio.
- Arquitectura final.
- Flujo end-to-end.
- Seguridad, eventos, consistencia y observabilidad.
- Integracion frontend.
- Evidencias principales.
- Aporte individual de cada integrante.
- Evidencia de participacion individual en GitHub.
- Demo asignada a cada integrante.
- Riesgos, incidencias y mejoras futuras.

Datos del equipo

Campo	Valor
Equipo	Pendiente de confirmar
Proyecto	ServiYa
Link GitHub	https://github.com/mary321mq/serviya-v2
Link documentacion	https://mary321mq.github.io/serviya-v2/
Rama integrada evaluada	<code>main</code>
Evidencia de merge	Historial de commits y rama <code>main</code> en GitHub

Guion de presentacion final

Duracion total sugerida: 10 minutos de exposicion, 5 minutos de demo tecnica y 3 minutos de preguntas.

1. Apertura

- Proyecto: ServiYa.
- Problema: gestion integral de servicios tecnicos a domicilio.
- Objetivo: conectar clientes, trabajadores/cotizadores, tecnicos y administradores con pagos, comprobantes y seguimiento.

2. Arquitectura final

Explicar:

- Frontend Angular.
- API Gateway.
- Keycloak.
- Eureka y Config Server.
- Microservicios de solicitudes, pagos, tecnicos, usuarios, resenas, asignaciones y notificaciones.
- Prometheus y Grafana.

3. Flujo end-to-end

1. Cliente crea solicitud.
2. Trabajador revisa y cotiza.
3. Cliente paga.
4. Sistema asigna tecnico.
5. Tecnico atiende.

6. Cliente califica.

7. Notificaciones acompañan cada cambio relevante.

4. Seguridad

Mostrar:

- Login con Keycloak.
- Roles por panel.
- Rutas protegidas en gateway.
- Token JWT enviado desde frontend.

5. Eventos y consistencia

Mostrar:

- Notificación de bienvenida.
- Nueva cotización.
- Pago pendiente.
- Pago exitoso.
- Solicitud aprobada.
- Resena pendiente.

6. Observabilidad

Mostrar en Grafana:

- Estado de servicios.
- Tráfico HTTP por segundo.
- Errores HTTP 4xx/5xx.
- Latencia HTTP p95.

7. Aportes individuales

Cada integrante debe decir:

- Que desarrollo.
- Que archivos o servicios toco.
- Que commits o PR lo evidencian.
- Que parte demostrara en vivo.

8. Cierre

- Riesgos actuales.
- Incidencias corregidas.
- Mejoras futuras.
- Confirmar que el producto esta documentado, ejecutable e integrado.

Demo tecnica end-to-end

Esta guia define una demo reproducible para la defensa.

Preparacion

1. Levantar Keycloak.
2. Levantar Config Server y Eureka.
3. Levantar Gateway.
4. Levantar microservicios necesarios.
5. Levantar frontend Angular.
6. Levantar Prometheus y Grafana.

Flujo principal

1. Cliente crea solicitud

Evidencia esperada:

- La solicitud se registra.
- Se muestra en "Mis solicitudes".
- Llega notificacion `REQUEST_CREATED` .

2. Trabajador cotiza

Evidencia esperada:

- El trabajador ve solicitudes pendientes.
- Envia cotizacion.
- El cliente recibe notificacion `QUOTE_READY` .
- La solicitud pasa a `COTIZADO_ESPERANDO_PAGO` .

3. Cliente paga

Evidencia esperada:

- Se crea intencion de pago.
- Se notifica `PAYMENT_PENDING` .
- Al confirmar pago se notifica `PAYMENT_SUCCESS` .
- El cliente puede revisar el comprobante.

4. Tecnico atiende

Evidencia esperada:

- Se asigna tecnico.
- El tecnico acepta trabajo.
- Cliente recibe `TECHNICIAN_ON_THE_WAY` .
- Tecnico termina trabajo.
- Cliente recibe `REVIEW_PENDING` .

5. Cliente califica

Evidencia esperada:

- Se registra la resena.
- La solicitud queda cerrada o completada.

Diagnostico durante la demo

Fallo	Donde mirar	Evidencia
401/403	Gateway, token JWT, roles Keycloak	Consola frontend, logs gateway.
404	Ruta gateway o controlador backend	Network tab y rutas documentadas.

Fallo	Donde mirar	Evidencia
500	Logs del microservicio	Stacktrace y metricas 5xx.
No aparecen notificaciones	notification-ms, gateway, JWT subject	Endpoint /notification-ms/api/v1/notifications/me .
Servicio no aparece en Grafana	Prometheus targets, Actuator	Consulta up .

Comandos utiles

```
curl http://localhost:18080/actuator/health
```



```
curl http://localhost:19091/api/v1/query?query=up
```



Evidencia GitHub

La rubrica exige que el aporte individual sea verificable en GitHub y este integrado en la rama comun.

Evidencia grupal

Evidencia	Donde mostrarla
Rama integrada evaluada	Branch principal o rama final U3.
Commits del equipo	Historial de commits.
Pull Requests o merges	Seccion Pull Requests o historial Git.
Resolucion de conflictos	Commits de merge o capturas.
Documentacion	Cambios en <code>docs/</code> y <code>mkdocs.yml</code> .

Evidencia individual

Cada integrante debe tener un anexo con:

- Nombre.
- Rol o responsabilidad.
- Rama de trabajo.
- Commits de codigo.
- Commits de documentacion.
- Pull Requests o merges.
- Evidencia de la demo en vivo.
- Evidencia de integracion en la rama comun.

Comandos utiles

```
git log --oneline --decorate --graph --all -20
```



```
git shortlog -sn
```



```
git log --author="NOMBRE" --oneline
```



Topics del repositorio

El documento oficial de la Escuela pide configurar topics estandarizados en todos los repositorios academicos.

Topics aplicados para ServiYa

`campus-juliaca
semestre-2026-1
linea-software
tipo-pi
dist
ciclo-5
grupo-2-serviya`

Si el docente exige tambien seccion, agregar el topic correspondiente, por ejemplo:

`seccion-g1`

Catalogo aplicado

Campo	Valor
Campus	campus-juliaca
Semestre	semestre-2026-1
Linea	linea-software
Tipo de proyecto	tipo-pi
Curso	dist - Desarrollo de Aplicaciones Distribuidas
Ciclo	ciclo-5

Campo	Valor
Grupo	grupo-2-serviya

Como agregarlos en GitHub

1. Entrar al repositorio.
2. Abrir la pagina principal del repo.
3. En la parte derecha, buscar el bloque "About".
4. Click en el icono de configuracion.
5. En "Topics", pegar los topics uno por uno o separados por espacios.
6. Marcar "Use your GitHub Pages website".
7. Guardar cambios.

Regla del documento oficial

Usar unicamente topics del catalogo oficial. No crear variantes, traducciones ni abreviaturas diferentes.

Rubrica S15

La nota se calcula con:

Puntuacion acumulada = suma de (Peso * Puntuacion obtenida)
Nota final = (Puntuacion acumulada / 30) * 20



Escala:

Nivel	Puntaje
Inicio	0
Proceso	1
Logro	2
Logro destacado	3

Matriz de evaluacion

Dimension	Peso	Para lograr destacado en ServiYa
Integracion del producto U3	2	Mostrar U1, U2 y U3 integrados en una rama comun, con aportes del equipo funcionando end-to-end.
Funcionamiento tecnico U3	2	Demostrar infraestructura, servicios, seguridad, eventos, consistencia, observabilidad, frontend y sistema externo.
Pruebas, evidencia y diagnostico	2	Ejecutar demo reproducible y diagnosticar fallos con datos, logs, metricas y trazas.

Dimension	Peso	Para lograr destacado en ServiYa
Aporte individual, GitHub y demo	2	Evidenciar código y documentación por integrante, integrados y demostrados en vivo.
Defensa técnica	1	Explicar decisiones de arquitectura con precisión y criterio técnico.
Presentación, documentación y reproducibilidad	1	Presentación clara, sitio MkDocs, PDF desde docs, enlace publicado y anexos completos.

Autoevaluación previa

Dimension	Peso	Puntaje esperado	Evidencia
Integración del producto U3	2	3	Rama común, build backend/frontend, flujo integrado.
Funcionamiento técnico U3	2	3	Demo completa con gateway, Keycloak, microservicios y observabilidad.
Pruebas, evidencia y diagnóstico	2	3	Capturas, logs, Prometheus/Grafana y errores controlados.
Aporte individual, GitHub y demo	2	3	Anexos por integrante y commits verificables.
Defensa técnica	1	3	Guión técnico y respuestas preparadas.

Dimension	Peso	Puntaje esperado	Evidencia
Presentacion/documentacion	1	3	MkDocs publicado, PDF y PPT entregados.

Evidencias que deben estar listas

- Capturas o enlaces del dashboard Grafana.
- Comandos de arranque y verificacion.
- Logs de servicios durante la demo.
- Endpoint o pantallas que prueben cada evento.
- Capturas de Pull Requests, merges o commits.
- Anexos individuales completos.

Checklist de evaluacion S15

Este checklist traduce la guia de defensa tecnica S15 a evidencias concretas para ServiYa.

Entregables grupales

Requisito	Evidencia en ServiYa	Estado
Documentacion en MkDocs	Sitio MkDocs en <code>docs/</code> y <code>mkdocs.yml</code>	Listo
PDF generado desde documentacion	Exportar desde navegador o pipeline de docs	Pendiente
Presentacion final	PPT o equivalente	Pendiente
Documentacion publicada	GitHub Pages u otra plataforma	Pendiente
Producto integrado y ejecutable	Rama comun del equipo	Pendiente
Anexos individuales	Un anexo por integrante	Pendiente

Evidencia tecnica minima

Criterio	Que demostrar
Arquitectura final	Frontend, gateway, Keycloak, Eureka, Config, microservicios, observabilidad.
Flujo end-to-end	Cliente crea solicitud, trabajador cotiza, cliente paga, tecnico atiende, cliente califica.
Seguridad	Login Keycloak, roles, rutas protegidas y JWT en gateway.

Criterio	Que demostrar
Comunicacion sincrona	Angular -> Gateway -> microservicios; Feign entre servicios.
Eventos y consistencia	Notificaciones de cotizacion, pago, asignacion y resena pendiente.
Observabilidad	Prometheus, Grafana, metricas HTTP, errores 4xx/5xx y latencia p95.
Diagnostico	Logs, respuestas HTTP, metricas y errores frecuentes.
Integracion frontend	Paneles cliente, tecnico, trabajador y admin consumiendo gateway.

Preguntas esperadas del docente

1. Que parte desarrollaste y como se integra con el sistema?
2. Que evidencia demuestra que tu aporte funciona?
3. Como diagnosticarias un fallo relacionado con tu componente?

Orden recomendado de exposicion

1. Presentar proyecto, equipo y repositorio.
2. Explicar problema de negocio.
3. Mostrar arquitectura final.
4. Ejecutar demo end-to-end.
5. Mostrar seguridad, eventos, consistencia y observabilidad.
6. Mostrar documentacion reproducible.
7. Mostrar evidencias individuales en GitHub.
8. Cada integrante demuestra su aporte.
9. Cerrar con riesgos, incidencias y mejoras futuras.

Anexo Individual - [Nombre del integrante]

1. Nombre del integrante y Rol/Responsabilidad

Campo	Detalle
Nombre del integrante	[Nombre completo]
Rol/Responsabilidad	[Backend / Frontend / DevOps / QA / Documentacion / Seguridad / Observabilidad]
Modulo principal trabajado	[Modulo o microservicio]
Periodo de trabajo	[Fechas]

2. Enlaces a ramas de trabajo, commits, merges y pull requests

Codigo

Tipo	Enlace	Descripcion
Rama de trabajo	[URL]	[Descripcion]
Commit 1	[URL]	[Descripcion]
Commit 2	[URL]	[Descripcion]
Pull Request	[URL]	[Resumen]
Merge a rama comun	[URL]	[Evidencia]

Documentacion

Tipo	Enlace	Descripcion
Rama de documentacion	[URL]	[Descripcion]
Commit de documentacion	[URL]	[Descripcion]
Pull Request de documentacion	[URL]	[Resumen]
Merge de documentacion	[URL]	[Evidencia]

3. Evidencia detallada de la parte del sistema que el alumno demostrara en vivo

Funcionalidad asignada

[Describir funcionalidad]

Pasos para reproducir

- 1. [Paso 1]
- 2. [Paso 2]
- 3. [Paso 3]
- 4. [Paso 4]

Endpoints o pantallas involucradas

Tipo	Ruta o pantalla	Descripcion
Frontend	[URL o ruta Angular]	[Descripcion]
Backend	[Endpoint REST]	[Descripcion]

Tipo	Ruta o pantalla	Descripcion
Base de datos	[Tabla]	[Descripcion]
Observabilidad	[Dashboard o query]	[Descripcion]

Datos de prueba

```
{  
  "campo": "valor"  
}
```

Resultado esperado

```
[Resultado esperado durante la defensa]
```

Evidencias

Evidencia	Link o captura
Captura 1	[URL o ruta]
Captura 2	[URL o ruta]
Video corto	[URL]
Log o metrica	[URL o query]

4. Evidencia de integracion en la rama comun

Integracion tecnica

Evidencia	Link
Pull request aprobado	[URL]
Merge a rama comun	[URL]
Commit final en rama comun	[URL]
Ejecucion exitosa del build	[URL o captura]
Evidencia en GitHub Actions, si aplica	[URL]

Integracion visual o funcional

Evidencia	Link o captura
Pantalla integrada en frontend	[URL o captura]
Endpoint integrado con gateway	[URL o captura]
Evidencia en Grafana/Prometheus/Loki	[URL o captura]

5. Declaracion individual

Yo, [Nombre del integrante], declaro que la evidencia registrada en este anexo corresponde a mi contribucion individual al proyecto ServiYa y que esta integrada en la rama comun del equipo.

Fecha:

[DD/MM/2026]



Anexo Individual - Integrante 1

Completar usando la plantilla `plantilla-anexo-individual.md`.

Anexo Individual - Integrante 2

Completar usando la plantilla `plantilla-anexo-individual.md`.

Anexo Individual - Integrante 3

Completar usando la plantilla `plantilla-anexo-individual.md`.

Anexo Individual - Integrante 4

Completar usando la plantilla `plantilla-anexo-individual.md`.

